BROCELIANDE TRIATHLON

GUIDE DU CODE ET DE LA GESTION DU SITE

À l'usage des bénévoles qui font vivre le site du Brocéliande Triathlon : comprendre une page web, gérer un site, et modifier le nôtre sans risque.

1. LES FONDAMENTAUX DU HTML

Lire et modifier n'importe quelle page web.

2. LES FONDAMENTAUX DE LA GESTION D'UN SITE

Domaine, hébergement, sécurité, obligations légales, entretien.

3. LE CODE DU SITE DU BT

Où trouver chaque élément, et des recettes pas à pas.

Version du 30 octobre 2026 · Source : guides/GUIDE_CODE.md, PDF refait avec npm run guide

SOMMAIRE

Partie 1 — Les fondamentaux du HTML

1.1 Ce qu'est une page web

1.2 Balises et éléments

1.3 Les attributs

1.4 Le squelette d'une page

1.5 Les balises à connaître

1.6 Structure et sens : la sémantique

1.7 Les liens et les chemins

1.8 Les images

1.9 Le CSS en bref

1.10 Le JavaScript en bref

1.11 L'accessibilité

1.12 Les caractères spéciaux

1.13 Les outils utiles

1.14 Les bonnes habitudes

Partie 2 — Les fondamentaux de la gestion d'un site

2.1 Les pièces d'un site en ligne

2.2 Site statique ou site à gestion de contenu

2.3 Versions et sauvegardes

2.4 Accès et sécurité

2.5 Les obligations légales (France)

2.6 Le contenu

2.7 Le référencement (être trouvé sur Google)

2.8 Vitesse et mesure d'audience

2.9 La routine d'entretien

2.10 Passer le relais

Partie 3 — Le code du site du BT

3.1 Trois règles avant tout

3.2 Comment une page est fabriquée

3.3 Où se trouve quoi

3.4 Anatomie d'une page : l'exemple de `le-club.njk`

3.5 Les quatre signes Nunjucks à connaître

3.6 Les briques prêtes à l'emploi (`macros.njk`)

3.7 Les classes CSS les plus utiles

3.8 Recettes pas à pas

3.9 Pièges fréquents

3.10 Pour aller plus loin

PARTIE 1 — LES FONDAMENTAUX DU HTML

1.1 CE QU'EST UNE PAGE WEB

Une page web est un **simple fichier texte** que le navigateur (Chrome, Firefox, Safari, Edge) lit et met en forme. Trois langages s'y partagent le travail :

Langage	Rôle	Image pour s'en souvenir
HTML	Le contenu et sa structure : titres, paragraphes, images, liens	Les murs et les pièces de la maison
CSS	L'apparence : couleurs, polices, tailles, disposition	La peinture et la décoration
JavaScript	Les comportements : menu qui s'ouvre, fenêtre, compte à rebours	L'électricité et les interrupteurs

On peut modifier un texte sans toucher à l'apparence, et changer une couleur sans toucher aux textes. Ces trois couches sont séparées.

1.2 BALISES ET ÉLÉMENTS

Le HTML entoure chaque contenu d'une **balise ouvrante** et d'une **balise fermante** :

```
<p>Le club s'entraîne le mardi soir.</p>
```

- `<p>` ouvre un paragraphe, `</p>` le ferme (avec une barre oblique).
- L'ensemble (balises et contenu) s'appelle un **élément**.
- Quelques balises n'ont pas de fermeture, car elles ne contiennent rien : `<img>` (image), `<br>` (retour à la ligne), `<input>` (champ de formulaire), `<meta>`.

Les éléments s'**emboîtent** comme des poupées russes. Le dernier ouvert est toujours le premier fermé :

```
<p>Rendez-vous au <b>stade</b> à 18h30.</p>      ✓ correct
<p>Rendez-vous au <b>stade</p> à 18h30.</b>      ✗ balises croisées
```

Une balise oubliée ou croisée est la première cause de page « cassée ». Elle peut décaler toute la mise en page qui suit.

1.3 LES ATTRIBUTS

Une balise ouvrante peut porter des **attributs**, qui la précisent. On écrit le nom, un signe égal, puis la valeur entre guillemets droits :

```
<a href="/contact/" class="lien-texte">Nous écrire</a>
<img src="/assets/img/stage.jpg" alt="Les BT au stage de printemps">
```

Attribut	Rôle
`href`	Adresse d'un lien
`src`	Fichier d'une image, d'une vidéo, d'un script
`alt`	Description d'une image, lue par les personnes aveugles et par Google
`class`	Nom(s) de style CSS à appliquer (plusieurs, séparés par des espaces)
`id`	Identifiant **unique** dans la page (sert aux ancres `#…`)
`lang`	Langue du texte (`fr`)

Toujours des guillemets **droits** (`"`), jamais des guillemets typographiques (`« »` ou `“ ”`) dans le code.

1.4 LE SQUELETTE D'UNE PAGE

Toute page HTML suit le même plan :

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Titre de l'onglet</title>
    <meta name="description" content="Résumé affiché par Google">
    <link rel="stylesheet" href="/assets/css/main.css">
  </head>
  <body>
    … tout ce qui s'affiche …
  </body>
</html>
```

- La partie **`<head>`** est invisible : elle contient le titre de l'onglet, le résumé pour les moteurs de recherche, le lien vers le CSS et les réglages d'affichage.
- La partie **`<body>`** contient tout ce que le visiteur voit.
- `charset="utf-8"` permet d'écrire les accents directement. `viewport` adapte la page aux téléphones.

1.5 LES BALISES À CONNAÎTRE

Balise	Usage
`<h1>` à `<h6>`	Titres, du plus important (`h1`) au moins important
`<p>`	Paragraphe
`<a href="…">`	Lien
`<img src="…" alt="…">`	Image
`<ul>`, `<ol>`, `<li>`	Liste à puces, liste numérotée, élément de liste
`<b>` ou `<strong>`, `<em>`	Gras, mise en valeur
` `	Retour à la ligne (à utiliser avec modération)
`<div>`, `<span>`	Boîtes neutres, pour la mise en page (bloc / en ligne)
`<header>`, `<nav>`, `<main>`, `<footer>`	En-tête, menu, contenu principal, pied de page
`<section>`, `<article>`	Partie d'une page, contenu autonome (une actualité)
`<figure>`, `<figcaption>`	Image avec sa légende
`<table>`, `<tr>`, `<th>`, `<td>`	Tableau de données (pas de mise en page)
`<form>`, `<label>`, `<input>`, `<button>`	Formulaire, étiquette, champ, bouton

1.6 STRUCTURE ET SENS : LA SÉMANTIQUE

Choisir la balise qui correspond au **sens** du contenu, et non à son apparence :

- **un seul `h1` par page** (le grand titre), puis des `h2` pour les parties, des `h3` à l'intérieur, sans sauter de niveau ;
- une liste s'écrit avec `<ul>` / `<li>`, pas avec des tirets et des `<br>` ;
- un bouton qui déclenche une action est un `<button>` ; un lien qui mène ailleurs est un `<a>`.

Pourquoi c'est important : les lecteurs d'écran des personnes aveugles naviguent de titre en titre, et Google comprend mieux une page bien structurée. Le site est aussi plus facile à maintenir.

1.7 LES LIENS ET LES CHEMINS

Forme	Exemple	Quand l'utiliser
Adresse complète	`https://www.klikego.com/…`	Lien vers un autre site
Chemin depuis la racine	`/contact/`	Lien vers une page du même site

Forme	Exemple	Quand l'utiliser
Ancre	`/evenements/#triathlon` ou `#resultats`	Lien vers un endroit précis d'une page (l'`id` de l'élément)
Courriel	`mailto:broceliandetriathlon@gmail.com`	Ouvre la messagerie du visiteur
Téléphone	`tel:+33600000000`	Appel direct depuis un téléphone

Un texte de lien doit dire où il mène : « Télécharger le règlement (PDF) » plutôt que « Cliquez ici ».

1.8 LES IMAGES

Format	Pour quoi
JPG	Photos
PNG	Logos, images avec transparence ou aplats de couleur
SVG	Icônes et logos dessinés (restent nets à toutes les tailles)
WebP, AVIF	Formats modernes, 30 à 50 % plus légers, créés automatiquement sur notre site

Une image trop lourde ralentit le site, surtout sur téléphone : 2 000 pixels de large suffisent largement. Chaque image utile a un texte `alt` qui la décrit en une phrase. Une image purement décorative a `alt=""` (vide, mais présent).

1.9 LE CSS EN BREF

Une règle CSS dit **à quoi** elle s'applique (le sélecteur) et **comment** (des propriétés et leurs valeurs) :

```
.surtitre {
  color: #b80d57;
  font-size: 14px;
  text-transform: uppercase;
}
```

- `.surtitre` vise tous les éléments qui portent `class="surtitre"`.
- Quand deux règles se contredisent, la plus précise gagne, et à précision égale, la dernière écrite. C'est la **cascade** (le « C » de CSS).
- Les **variables** (`--orange: #ff891c;`) définissent une valeur une seule fois pour tout le site.
- Les **media queries** (`@media (max-width: 759px) { … }`) appliquent des règles seulement sur petit écran : c'est ce qui rend un site adapté au téléphone (*responsive*).
- Unités courantes : `px` (pixels), `%` (pourcentage du parent), `rem` (taille du texte de base), `vw` (pourcentage de la largeur de l'écran).

1.10 LE JAVASCRIPT EN BREF

Le JavaScript ajoute de l'interactivité : ouvrir un menu, afficher une fenêtre, calculer un compte à rebours. Règle d'or : **le contenu essentiel doit rester lisible sans JavaScript**. Certains visiteurs ou moteurs de recherche ne l'exécutent pas, et une erreur dans le script ne doit pas cacher les informations du club.

1.11 L'ACCESSIBILITÉ

Un site accessible est utilisable par tous : personnes aveugles ou malvoyantes, personnes qui naviguent au clavier, visiteurs sur un petit écran en plein soleil. Les bases :

- un texte `alt` pour chaque image utile ;
- un **contraste** suffisant entre le texte et son fond (pas de gris clair sur blanc) ;
- un **focus visible** : la touche Tab doit montrer où l'on se trouve ;
- une **étiquette** (`<label>`) pour chaque champ de formulaire ;
- des liens explicites, des titres dans l'ordre, la langue déclarée (`lang="fr"`) ;
- pas d'information donnée **uniquement** par la couleur.

La référence est le **WCAG** (et sa déclinaison française, le **RGAA**). Ces règles sont obligatoires pour les organismes publics et certaines entreprises ; pour un club, c'est une bonne pratique qui profite à tous.

1.12 LES CARACTÈRES SPÉCIAUX

- Les accents s'écrivent directement grâce à l'encodage **UTF-8**.
- Trois caractères ont un sens particulier en HTML. Pour les afficher tels quels : `&` pour `&`, `<` pour `<`, `>` pour `>`.
- ` ` est une **espace insécable**, qui empêche un retour à la ligne (entre « 23 » et « mai », avant « : »). Sur notre site, elles sont ajoutées automatiquement.

1.13 LES OUTILS UTILES

- **Un éditeur de code**, par exemple Visual Studio Code (gratuit). Il colore le code et signale les balises mal fermées. Sur notre site, pas besoin de l'installer : le crayon de github.com suffit pour une petite modification, et la touche **.** (point) ouvre Visual Studio Code dans le navigateur. Évite Word et les traitements de texte, qui ajoutent des caractères invisibles.
- **L'inspecteur du navigateur** : clic droit sur un élément de la page, puis « Inspecter » (ou la touche F12). Il montre le HTML et le CSS de l'élément, et permet d'essayer une modification en direct, sans risque : tout disparaît au rechargement de la page.
- **Les contrôleurs** : le validateur du W3C (validator.w3.org) pour le HTML, Lighthouse (dans Chrome, onglet « Lighthouse » de l'inspecteur) pour la vitesse, l'accessibilité et le référencement. Sur notre site, chaque publication contrôle le HTML et les liens internes (onglet Actions) ; `npm run check`, sur un ordinateur équipé, ajoute l'accessibilité.

1.14 LES BONNES HABITUDES

- **Une modification à la fois**, puis vérification. En cas de problème, on sait d'où il vient.

- **Indenter** le code (décaler les éléments emboîtés) pour voir la structure d'un coup d'œil.
- **Commenter** les blocs (`<!-- commentaire -->` en HTML, `{# … #}` sur notre site) pour s'y retrouver plus tard.
- **Vérifier sur téléphone**, pas seulement sur ordinateur : une grande partie des visiteurs consultent le site depuis leur mobile.
- **Garder une trace** de chaque modification (sur notre site : un enregistrement Git avec un résumé clair).

PARTIE 2 – LES FONDAMENTAUX DE LA GESTION D'UN SITE

2.1 LES PIÈCES D'UN SITE EN LIGNE

Pièce	Ce que c'est	Pour le BT
Nom de domaine	L'adresse du site (`broceliandetri.org`). Il se **loue** à l'année auprès d'un bureau d'enregistrement (*registrar*).	À reprendre en main auprès de Wix (voir le README)
DNS	L'annuaire d'internet, qui relie le nom de domaine au serveur qui héberge le site. Se règle chez le *registrar*.	À régler au moment de la mise en ligne
Hébergement	L'ordinateur, toujours allumé, qui stocke les fichiers et les envoie aux visiteurs.	GitHub Pages (gratuit)
Certificat HTTPS	Le cadenas dans la barre d'adresse : les échanges sont chiffrés. Indispensable aujourd'hui.	Fourni gratuitement par GitHub Pages
Messagerie	Les adresses e-mail, indépendantes du site.	Gmail, non concernée par le site

Si le nom de domaine n'est pas renouvelé, le site devient inaccessible et l'adresse peut être rachetée par quelqu'un d'autre. **Active le renouvellement automatique** et note la date d'échéance.

2.2 SITE STATIQUE OU SITE À GESTION DE CONTENU

- Un **site à gestion de contenu** (WordPress, Wix…) fabrique chaque page à la demande, à partir d'une base de données. On le modifie dans une interface en ligne. En contrepartie, il demande des mises à jour de sécurité régulières, ou un abonnement.
- Un **site statique**, comme celui du BT, est fabriqué à l'avance : ce sont de simples fichiers. Il est rapide, très difficile à pirater (rien à attaquer côté serveur) et gratuit à héberger. En contrepartie, on le modifie en éditant des fichiers, ce que ce guide explique.

2.3 VERSIONS ET SAUVEGARDES

- **Git** garde l'historique de chaque modification : qui, quand, quoi. On peut comparer deux versions, ou revenir en arrière en un clic (voir le guide de gestion). **GitHub** conserve cet historique en ligne.
- Ce qui n'est pas dans Git doit être **sauvegardé ailleurs** : photos originales en haute définition, documents sources (affiches, check-list), fichiers reçus des partenaires.
- Bonne règle : **trois copies, sur deux supports différents, dont une hors du club**. Par exemple l'ordinateur du gestionnaire, un disque externe, et le Drive du club.

2.4 ACCÈS ET SÉCURITÉ

- **Des comptes au nom du club**, pas d'une personne : une organisation GitHub, une adresse e-mail du club. Le jour où un bénévole passe la main, le site ne part pas avec lui.
- **Au moins deux administrateurs** pour chaque compte important (GitHub, nom de domaine, Facebook, Instagram).
- **Des mots de passe longs et différents** pour chaque service, rangés dans un gestionnaire de mots de passe (Bitwarden, 1Password, celui du navigateur…).
- **La double authentification** (code reçu sur le téléphone ou application dédiée) sur GitHub, sur le compte du nom de domaine et sur la messagerie.
- **Une liste des accès** (quel compte, quel e-mail, qui est administrateur), conservée par le bureau. Elle ne contient pas les mots de passe eux-mêmes.
- **Jamais de mot de passe, de clé ou de donnée personnelle dans le dépôt** : le dépôt GitHub est public, tout le monde peut le lire.
- Quand un bénévole quitte le bureau : **retirer ses accès**, et changer les mots de passe qu'il connaissait.

2.5 LES OBLIGATIONS LÉGALES (FRANCE)

Obligation	Ce qu'elle impose	Sur notre site
Mentions légales (loi pour la confiance dans l'économie numérique)	Identifier l'éditeur (nom de l'association, siège), le directeur de la publication et l'hébergeur (nom, adresse, téléphone)	Page Mentions légales, complète (informations dans `src/_data/legal.json`)
RGPD (données personnelles)	Dire quelles données sont collectées, pourquoi, combien de temps, et comment exercer ses droits ; ne collecter que le nécessaire	Page Confidentialité, complète ; le formulaire de contact ne demande que le nom, l'e-mail et le message
Cookies et traceurs (règles de la CNIL)	Accord **préalable** du visiteur avant tout cookie non indispensable. Refuser doit être aussi simple qu'accepter, et le choix peut être retiré.	Aucun cookie du site ; bandeau d'accord pour Facebook et Instagram
Droit d'auteur	Une photo, un texte ou un logo trouvé sur internet n'est pas libre : il faut l'accord de l'auteur et citer le crédit	Photos de presse : accord d'Ouest-France à obtenir avant la mise en ligne
Droit à l'image	Accord des personnes reconnaissables sur une photo ; pour un mineur, accord de ses parents	Conseil : prévoir une autorisation de publication des photos dans le bulletin d'adhésion

2.6 LE CONTENU

- **À jour avant tout** : une date passée, un tarif de l'an dernier ou un lien mort font plus de tort qu'une page simple. Retire ce qui est obsolète.
- **Une information à un seul endroit** : sur notre site, les horaires sont dans `semaine.json`, les tarifs dans `tarifs.json`. On les modifie une fois, et toutes les pages suivent.

- **Concret et local** : un lieu, un horaire, un nom, un résultat. Évite les formules creuses (« notre passion », « une communauté dynamique »).
- **Relire** chaque ajout, ou le faire relire, avant de l'envoyer en ligne.
- **Dater** les actualités, pour que le visiteur sache si l'information est récente.

2.7 LE RÉFÉRENCEMENT (ÊTRE TROUVÉ SUR GOOGLE)

- Un **titre** et une **description** uniques pour chaque page, qui disent ce qu'on y trouve (ville, sport, public).
- Du **contenu concret** : les mots que les gens tapent (« triathlon Montfort-sur-Meu », « club triathlon Brocéliande »).
- Des **liens entrants** : page Facebook, site de la mairie, de la FFTRI, de la ligue, des partenaires. Demande-leur de pointer vers le site.
- Une **fiche Google Business Profile** du club (gratuite) : elle apparaît dans Google Maps et dans les résultats de recherche locaux.
- La **Google Search Console** (gratuite) : on y déclare le site et son plan (`sitemap.xml`), et elle signale les erreurs.
- Des **adresses stables** : ne renomme pas une page déjà en ligne (`/le-club/`), sinon les liens et les résultats Google qui y mènent seront cassés.

2.8 VITESSE ET MESURE D'AUDIENCE

- **Vitesse** : des images légères, peu de scripts venus d'autres sites, des polices hébergées avec le site. Contrôle régulier avec Lighthouse ou PageSpeed Insights (pagespeed.web.dev).
- **Mesure d'audience** : notre site n'en a pas, ce qui évite tout bandeau de cookies pour cela. S'il en faut une un jour, choisir un outil que la CNIL dispense de consentement quand il est bien configuré (par exemple Matomo), et mettre à jour la page Confidentialité.

2.9 LA ROUTINE D'ENTRETIEN

Quand	Quoi
Après chaque modification	Coche verte dans l'onglet Actions ; vérifier la page en ligne sur ordinateur **et** sur téléphone
Chaque mois	Agenda (retirer les dates passées), actualités, publications Facebook ; un tour rapide des pages
Chaque trimestre	Tester les liens vers les autres sites (Klikego, partenaires, réseaux) ; contrôler que la publication sur GitHub fonctionne (onglet Actions)
À la rentrée	Tarifs, horaires, documents de la saison, bureau, tenues ; relire les pages légales
Chaque année	Renouvellement du nom de domaine ; mise à jour des outils (voir le guide de gestion) ; revue des accès et des administrateurs ; vérification des sauvegardes

2.10 PASSER LE RELAIS

Un site de club change de mains au fil des saisons. Pour que la transmission se passe bien :

- garder ces guides à jour quand une façon de faire change ;
- tenir la liste des accès et des comptes ;
- former un deuxième bénévole aux gestes courants (modifier un texte, publier) ;
- noter dans l'historique Git un résumé clair de chaque modification.

PARTIE 3 — LE CODE DU SITE DU BT

3.1 TROIS RÈGLES AVANT TOUT

1. **On modifie toujours le dossier `src/`.** Le site *fabriqué* (dossier `dist/`) est entièrement recréé à chaque publication : il n'apparaît d'ailleurs pas dans le dépôt sur github.com. Sur un ordinateur, ne modifie jamais `dist/` : toute modification y serait perdue.
2. **Les listes sont dans les données, les textes fixes dans les pages.**
   - Un partenaire, un article, un tarif, un rendez-vous ou une tenue se changent dans `src/_data/*.json` (voir le README).
   - Un titre, un paragraphe ou la disposition d'une page se changent dans `src/*.njk`.
3. **Enregistre, puis vérifie.**
   - Sur github.com, chaque modification enregistrée (« Commit changes ») est contrôlée puis publiée en deux à trois minutes. Coche verte dans l'onglet Actions : regarde la page en ligne. Croix rouge : l'ancienne version reste en ligne, et le message indique le fichier et la ligne en faute (guide de gestion, section 4.4).
   - Pour une modification de mise en page, mieux vaut voir le résultat avant de publier : sur un ordinateur équipé de Node.js (guide de gestion, section 3.8), `npm run dev` affiche le site sur http://localhost:8080, et `npm run check` contrôle tout, accessibilité comprise.

3.2 COMMENT UNE PAGE EST FABRIQUÉE



Chaque fichier `.njk` contient **du HTML ordinaire**, plus quelques balises **Nunjucks** entre accolades. Ces balises insèrent des données, répètent un bloc pour chaque élément d'une liste, ou appellent une brique toute prête. L'outil **Eleventy** assemble le tout et écrit les pages dans `dist/`.

Ce qu'Eleventy fait automatiquement à chaque génération, sans rien à écrire :

- il crée les versions légères des images (AVIF, WebP, plusieurs tailles) ;
- il ajoute les espaces insécables de la typographie française (avant `: ; ! ?`, entre un nombre et son unité, etc.) ;
- il ouvre les liens vers les autres sites dans un nouvel onglet, en le signalant aux lecteurs d'écran ;
- il ajoute le menu, l'en-tête et le pied de page sur chaque page.

3.3 OÙ SE TROUVE QUOI

Je veux modifier…	Fichier
L'accueil	`src/index.njk`
Le club (histoire, chiffres, bureau, équipement)	`src/le-club.njk`
Entraînements	`src/entrainements.njk`, et la semaine type dans `src/_includes/composants/semaine-type.njk`
Vie du club	`src/vie-du-club.njk`
Nos évènements	`src/evenements.njk`
Nous rejoindre	`src/nous-rejoindre.njk`
Partenaires, Devenir partenaire	`src/partenaires/index.njk`, `src/partenaires/devenir-partenaire.njk`
Contact	`src/contact.njk`
Mentions légales, Confidentialité	`src/mentions-legales.njk`, `src/confidentialite.njk` ; les informations du club (RNA, siège, directeur de la publication…) dans `src/_data/legal.json`
Page d'erreur (adresse inconnue)	`src/404.njk`
En-tête et menu	`src/_includes/composants/en-tete.njk` (les entrées du menu sont dans `src/_data/navigation.json`)
Pied de page	`src/_includes/composants/pied-de-page.njk` (liens dans `navigation.json`)
Bandeau d'accord Facebook/Instagram	`src/_includes/composants/bandeau-meta.njk`
Squelette commun (balises `<head>`, polices, scripts)	`src/_includes/layouts/base.njk`
Les briques réutilisables (bouton, bandeau, cartes…)	`src/_includes/macros.njk`
Couleurs, polices, mise en forme	`src/assets/css/main.css`
Comportements (menu, frise, fenêtres, Facebook…)	`src/assets/js/main.js`
Images, PDF, vidéo	`src/assets/img/`, `src/assets/docs/`, `src/assets/video/`

3.4 ANATOMIE D'UNE PAGE : L'EXEMPLE DE `le-club.njk`

```
---
title: Le club
description: "Le Brocéliande Triathlon, le BT pour les intimes, c'est avant tout un club…"
permalink: /le-club/
---
{%- import "macros.njk" as m with context -%}

{% call m.bandeau(surtitre="Le club", titre="Bienvenue au BT !", deuxColonnes=true) %}
  <p>Le Brocéliande Triathlon, le BT pour les intimes…</p>
{% endcall %}

{# Histoire #}
<section class="section">
  <div class="conteneur py-72 deux-colonnes deux-colonnes--centre">
    <div class="pile g-14">
      <p class="surtitre">Depuis le 27 septembre 2008</p>
      <h2 class="titre t-52">L'histoire du club</h2>
      <p>Le club est né pour faire découvrir…</p>
    </div>
    <img src="/assets/img/club_membres_2026.jpg" alt="Les membres du club réunis pour la photo de
groupe 2026">
  </div>
</section>
```

Ligne par ligne :

1. **L'en-tête, entre les deux lignes** `---`, contient les réglages de la page :
   - `title` : le titre de l'onglet du navigateur et des résultats Google. « · Brocéliande Triathlon » est ajouté automatiquement.
   - `description` : le résumé affiché par Google, unique pour chaque page, environ 150 caractères.
   - `permalink` : l'adresse de la page. **Ne la change pas** : les liens qui y mènent seraient cassés.
   - `contenusMeta: true` (accueil et Vie du club seulement) : la page affiche Facebook ou Instagram, donc le bandeau d'accord.
2. **`{%- import "macros.njk" as m with context -%}`** rend les briques disponibles sous le préfixe `m.` (par exemple `m.bouton(…)`). Garde cette ligne en haut de chaque page.
3. **`{% call m.bandeau(…) %} … {% endcall %}`** crée le grand bandeau noir du haut, avec le titre principal de la page (le seul `h1`). Le texte placé entre les deux balises s'affiche dans le bandeau.
4. **`{# Histoire #}`** est un commentaire : il n'apparaît pas sur le site. Chaque bloc de page en a un, pour s'y retrouver.
5. **`<section class="section">`** ouvre un bloc de la page. À l'intérieur, **`<div class="conteneur …">`** centre le contenu à la bonne largeur. Toute nouvelle section reprend ce duo.

6. **Le reste est du HTML ordinaire.** Les classes (`py-72`, `pile`, `titre t-52`…) règlent l'apparence (voir la section 3.7).

3.5 Les quatre signes Nunjucks à connaître

Signe	Rôle	Exemple
`{{ … }}`	**Affiche** une donnée	`{{ tenues.saison }}` affiche « 2026 – 2027 »
`{% … %}`	**Commande** : boucle, condition, appel de brique	`{% for tarif in tarifs %} … {% endfor %}`
`{# … #}`	**Commentaire**, invisible sur le site	`{# Documents #}`
`|`	**Filtre** : transforme une donnée	`{{ agenda | exclure("accueil", false) }}`

Une boucle affiche le même modèle pour chaque élément d'une liste du dossier `_data` :

```
<ul class="grille min-250 g-14">
  {%- for tarif in tarifs %}
  <li class="tarif">
    <span class="tarif__prix">{{ tarif.prix }}</span>
    <h3 class="tarif__formule">{{ tarif.formule }}</h3>
  </li>
  {%- endfor %}
</ul>
```

Pour ajouter un tarif, **ne touche pas à ce code** : ajoute un bloc dans `src/_data/tarifs.json`, et la boucle l'affiche.

Une condition n'affiche un bloc que si une donnée existe :

```
{%- if tri.parcours.afficher %} … {%- endif %}
```

Le tiret dans `{%-` sert seulement à supprimer les lignes vides dans le HTML final. Tu peux le laisser tel quel.

3.6 Les briques prêtes à l'emploi (`macros.njk`)

Elles s'utilisent dans n'importe quelle page, grâce à la ligne `import` du haut.

Brique	À quoi elle sert	Exemple
`m.bouton(texte, adresse, variante, classes)`	Bouton arrondi. `variante` : `"orange"`, `"noir"`, `"contour"` (fond sombre) ou `"contour-noir"` (fond clair). Sans adresse (`""`), le bouton est visible mais inactif.	`{{ m.bouton("Nous contacter", "/contact/", "noir") }}`
`m.bandeau(surtitre, titre, deuxColonnes)`	Bandeau noir du haut de page (avec `{% call %}`)	voir la section 3.4
`m.icone(nom, taille)`	Petite icône : `instagram`, `facebook`, `youtube`, `strava`, `calendrier`, `sac`, `document`, `ppg`, `menu`, `fermer`	`{{ m.icone("calendrier", 24) }}`
`m.friseAgenda(rendez-vous)`	Frise de l'agenda (horizontale ou verticale selon la place)	`{{ m.friseAgenda(agenda) }}`
`m.compteARebours(date)`	Pastille « J-xxx »	`{{ m.compteARebours(evenements.triathlon.depart) }}`
`m.penseBete()`	Pense-bête de la check-list (Nous rejoindre)	`{{ m.penseBete() }}`
`m.carteActu`, `m.cartePresse`, `m.cartePartenaire`, `m.carteResultat`, `m.carteDiscipline`	Les cartes des listes (actus, presse, partenaires, résultats, disciplines)	utilisées dans les boucles
`m.blocFacebook`, `m.contenuFacebook`, `m.contenuInstagram`	Vignettes et publications Facebook/Instagram intégrées, avec l'accord du visiteur	à laisser tel quel

3.7 LES CLASSES CSS LES PLUS UTILES

Elles s'ajoutent dans l'attribut `class="…"`, séparées par des espaces. Toutes sont définies dans `src/assets/css/main.css`.

Mise en page

Classe	Effet
`conteneur`	Centre le contenu à la largeur du site (1240 px), avec des marges sur les côtés
`fond-blanc`, `fond-noir`, `fond-rose`	Couleur de fond d'une section (sans classe : fond crème)
`py-80`, `py-72`, `py-64`, `py-56`, `py-48`	Espace en haut et en bas de la section (en pixels)
`pt-64`, `pb-72` …	Espace en haut seulement (`pt`) ou en bas seulement (`pb`)
`pile` + `g-16`	Éléments empilés les uns sous les autres, espacés de 16 px (`g-4` à `g-32`)
`grille min-260 g-16`	Grille de cartes qui passe toute seule à la ligne : chaque carte fait au moins 260 px (`min-150` à `min-440`)
`deux-colonnes`	Deux colonnes côte à côte, empilées sur téléphone
`section__tete`	Ligne de titre de section, avec un lien ou un bouton à droite

Textes

Classe	Effet
`titre t-52`	Titre en capitales (police Anton), taille 52 px (de `t-22` à `t-72`)
`surtitre`	Petite ligne en capitales roses au-dessus d'un titre (`surtitre--sombre` : orange, pour les fonds noirs)
`texte-gris`, `texte-blanc`, `texte-orange`, `texte-600`	Couleur ou graisse du texte
`mesure-52`	Limite la longueur des lignes pour faciliter la lecture (52 caractères)
`liste-puces`	Liste à puces classique
`lien-fort`	Lien en capitales souligné de rose (`lien-fort--orange` : trait orange, `lien-fort--clair` : texte blanc)
`lien-texte`	Lien en gras dans un paragraphe
`sr-only`	Texte invisible, lu seulement par les lecteurs d'écran

Images

Classe	Effet
`image-pleine`	L'image prend toute la largeur disponible
`ombre-orange`, `ombre-rose`	Ombre décalée de couleur, comme sur les photos de l'accueil et du club

3.8 RECETTES PAS À PAS

Toutes ces recettes se font sur github.com : le crayon pour modifier un fichier, **Add file → Upload files** pour envoyer une image ou un PDF, la touche **.** (point) pour modifier plusieurs fichiers d'un coup (guide de gestion, section 4). Dans l'éditeur, `Ctrl` + `F` cherche un mot.

MODIFIER UN TEXTE

Ouvre la page `.njk`, cherche la phrase avec `Ctrl` + `F`, et change seulement le texte entre `>` et `<`. Ne supprime ni les balises (`<p>`, `</p>`…) ni les accolades.

AJOUTER UN PARAGRAPHE

```
<p>Le nouveau texte du paragraphe.</p>
```

Pour mettre un mot en gras : `<b>mot</b>`. Pour un lien vers une autre page du site : `<a class="lien-texte" href="/contact/">nous écrire</a>`.

AJOUTER UN BOUTON

```
{{ m.bouton("Voir le planning", "/entrainements/", "orange") }}
```

Plusieurs boutons côte à côte se placent dans `<div class="boutons"> … </div>`.

AJOUTER UNE IMAGE

1. Dépose la photo dans `src/assets/img/`. Nom sans espace ni accent (`stage-2027.jpg`), 2 000 px de large au plus.
2. Dans la page :

```
<img class="image-pleine" src="/assets/img/stage-2027.jpg" alt="Les BT au stage de Pâques 2027, au bord du lac">
```

Le texte `alt` décrit la photo pour les personnes aveugles. Pour une image purement décorative, mets `alt=""`. Les versions légères de l'image sont créées automatiquement.

AJOUTER UNE SECTION COMPLÈTE

Copie ce modèle à l'endroit voulu, entre deux sections existantes :

```
{# Stage de printemps #}
<section class="section fond-blanc">
  <div class="conteneur py-64 pile g-16">
    <p class="surtitre">Avril 2027</p>
    <h2 class="titre t-44">Stage de printemps</h2>
    <p class="mesure-62">Trois jours d'entraînement en groupe autour du lac de Trémelin.</p>
    {{ m.bouton("Nous contacter", "/contact/", "noir") }}
  </div>
</section>
```

Garde la hiérarchie des titres : `h2` pour un titre de section, `h3` à l'intérieur d'une section. Le `h1` est réservé au bandeau du haut.

MASQUER UN BLOC SANS LE SUPPRIMER

Entoure-le de `{#` et `#}` :

```
{#
<section class="section"> … </section>
#}
```

Pour le faire réapparaître, retire simplement ces deux marques. Attention : un bloc masqué ne doit pas lui-même contenir de commentaire `{# … #}`.

LIEN VERS UN DOCUMENT PDF

Dépose le PDF dans `src/assets/docs/` (par exemple `stage-2027.pdf`), puis :

```
<a class="lien-texte" href="/assets/docs/stage-2027.pdf">Programme du stage (PDF)</a>
```

Pour les documents de la page Nous rejoindre, passe plutôt par `src/_data/documents.json`.

AJOUTER UNE PAGE

1. Copie une page simple, par exemple `src/mentions-legales.njk`, sous un nouveau nom : `src/stages.njk`. Sur github.com : ouvre la page, icône **Copy raw file**, puis dans le dossier `src`, **Add file → Create new file**, tape le nom `stages.njk` et colle le contenu.
2. Change l'en-tête : `title`, `description` et `permalink: /stages/`.
3. Remplace le contenu.
4. Pour la mettre dans le menu, ajoute `{ "texte": "Stages", "url": "/stages/" }` dans `src/_data/navigation.json`, liste `principale`. Le menu compte déjà 6 entrées : au-delà de 7, il devient difficile à lire.
5. Pour l'inclure dans le fichier de relecture (`npm run html`), ajoute aussi la page dans la liste `PAGES` de `scripts/fichier-html.mjs`.

CHANGER UNE COULEUR DU CLUB

Les couleurs sont définies une seule fois, au début de `src/assets/css/main.css` (section « 2. Variables »). Par exemple, `--orange: #ff891c;`. Changer la valeur change cette couleur partout sur le site. Vérifie ensuite le contraste : `npm run check` sur un ordinateur équipé, ou PageSpeed Insights (pagespeed.web.dev, rubrique Accessibilité) sur la page en ligne.

3.9 PIÈGES FRÉQUENTS

Symptôme	Cause probable	Solution
La publication échoue (croix rouge dans l'onglet Actions) avec le message `Template render error` et un numéro de ligne	Une balise Nunjucks est mal fermée : `{% for %}` sans `{% endfor %}`, `{% if %}` sans `{% endif %}`, `{{` sans `}}`	Va à la ligne indiquée et vérifie les paires
Une partie de la page est mal placée ou a disparu	Une balise HTML n'est pas fermée (`<div>` sans `</div>`)	La publication échoue à l'étape « Vérifier le site » : le message indique le fichier et la ligne
Erreur dans un appel de brique	Une apostrophe ferme le texte trop tôt : `m.bouton('L'équipe', …)`	Entoure toujours les textes de guillemets droits doubles : `m.bouton("L'équipe", …)`
Une modification n'apparaît pas	La publication n'est pas finie ou a échoué ; la page est restée en mémoire dans le navigateur ; sur un ordinateur, le fichier a été modifié dans `dist/`	Regarde l'onglet Actions, puis recharge la page (Ctrl + F5). Sur un ordinateur, modifie `src/` et regarde le terminal de `npm run dev`
Une image n'apparaît pas	Nom du fichier différent (majuscule, accent, extension `.JPG` / `.jpg`)	Recopie exactement le nom du fichier
Un lien mène à une page introuvable	Adresse mal écrite : il manque le `/` final (`/contact` au lieu de `/contact/`)	La publication échoue et liste les liens cassés
`npm run check` ou PageSpeed Insights signale un contraste insuffisant (« color-contrast »)	Texte trop pâle sur son fond	Utilise les couleurs prévues : texte rose foncé (`--rose-fonce`) sur fond clair, orange sur fond noir

3.10 POUR ALLER PLUS LOIN

- **CSS (`src/assets/css/main.css`)** : le fichier est découpé en sections numérotées (« 10. Accueil », « 11. Le club »…). Les règles d'une page se trouvent dans sa section.
- **JavaScript (`src/assets/js/main.js`)** : sections numérotées de 1 à 8 (menu, semaine type, compte à rebours, vidéo, formulaire, Facebook/Instagram, frise, fenêtres). Il n'y a normalement rien à y modifier pour mettre à jour les contenus.
- **Scripts (`scripts/`)** : la fabrication du PDF de la check-list, le recadrage des tenues, le fichier de relecture et les contrôles. Ils se lancent avec les commandes `npm run …` listées dans le README, sur un ordinateur

équipé de Node.js. Le recadrage des tenues et la liste des textes entre crochets tournent aussi tout seuls à chaque publication.